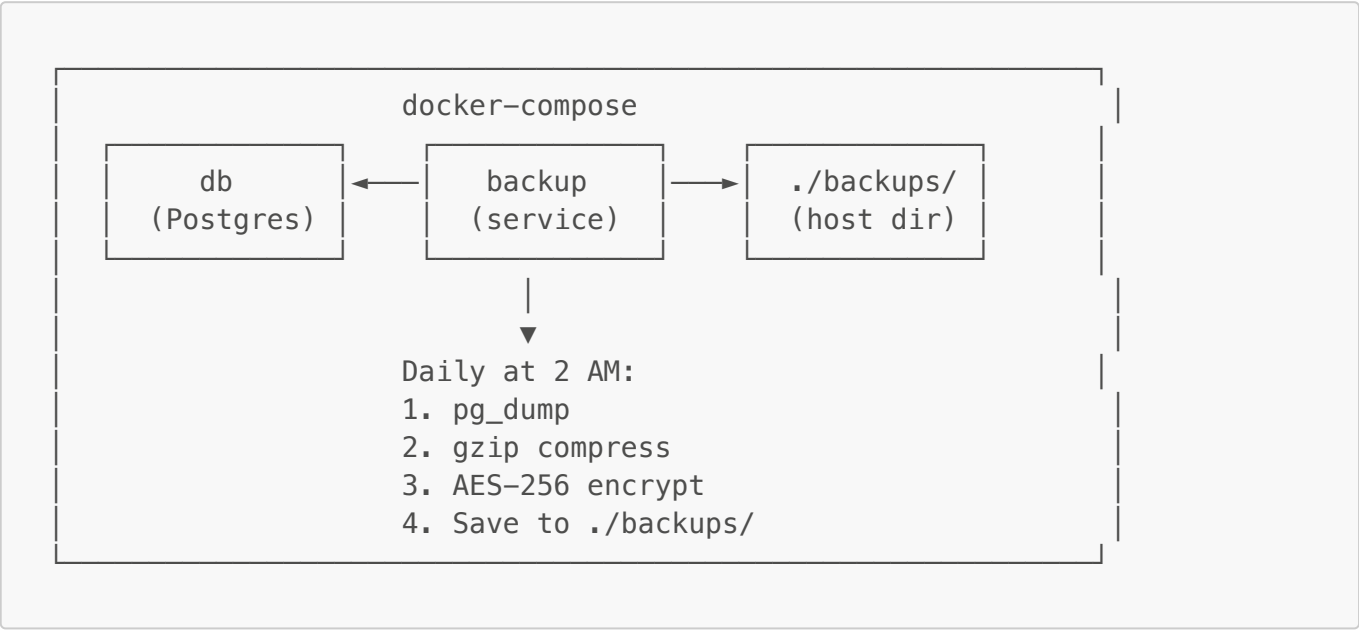


Backup & Recovery System

This document explains how the automated backup system works and how to use it.

How It Works

Architecture



The Backup Process (Step by Step)

- 1. **Scheduling:** The backup container runs continuously and calculates seconds until 2 AM.
- 2. **Database Dump:** At 2 AM, it runs `pg_dump` to export all database tables.
- 3. **Compression:** The SQL dump is piped through `gzip` to reduce size (~90% smaller).
- 4. **Encryption:** The compressed data is encrypted using `openssl` with AES-256-CBC.
- 5. **Storage:** The encrypted file is saved to `./backups/backup_YYYYMMDD_HHMMSS.sql.gz.enc`.
- 6. **Rotation:** Backups older than **6 years** are automatically deleted (HIPAA minimum).

Encryption Details

Setting	Value
Algorithm	AES-256-CBC
Key Derivation	PBKDF2
Key Source	<code>BACKUP_ENCRYPTION_KEY</code> in <code>.env</code>

Important: Without the encryption key, backups cannot be restored!

Quick Start

- 1. Set Your Encryption Key

Edit `.env` and set a strong, unique key:

```
BACKUP_ENCRYPTION_KEY=your_super_secret_key_here
```

IMPORTANT: Store this key securely! If lost, you cannot restore backups.

2. Start the Backup Service

```
docker-compose up -d backup
```

3. Verify It's Running

```
docker logs -f caregiver_backup
```

You should see:

```
Backup service started. Running daily at 2 AM.  
Next backup in 28374 seconds...
```

Manual Operations

Create an Immediate Backup

```
./scripts/manual_backup.sh
```

Output:

```
=====
MANUAL DATABASE BACKUP
=====
Database: caregiver_communications_platform_database
Output: ./backups/manual_backup_20260118_153500.sql.gz.enc

Creating backup...

✅ Backup completed!
File: ./backups/manual_backup_20260118_153500.sql.gz.enc
Size: 1.2M
```

Restore a Backup

```
./scripts/restore_backup.sh ./backups/backup_20260118_020000.sql.gz.enc
```

Output:

```
=====
DATABASE RESTORE
=====
Backup file: ./backups/backup_20260118_020000.sql.gz.enc
Target database: caregiver_communications_platform_database

This will OVERWRITE the current database. Continue? (yes/no): yes

Decrypting and restoring...

✅ Restore completed successfully!
```

File Reference

File	Purpose
docker-compose.yml	Contains the backup service definition
.env	Contains BACKUP_ENCRYPTION_KEY
./backups/	Directory where encrypted backups are stored
scripts/manual_backup.sh	Create an immediate backup
scripts/restore_backup.sh	Restore from a backup file

Backup File Naming

Pattern	Example	Source
backup_*.sql.gz.enc	backup_20260118_020000.sql.gz.enc	Automated daily
manual_backup_*.sql.gz.enc	manual_backup_20260118_153500.sql.gz.enc	Manual script

Recovery Scenarios

Scenario 1: Corrupted Database

```
# Stop the server
docker-compose stop server

# Restore latest backup
./scripts/restore_backup.sh ./backups/backup_20260118_020000.sql.gz.enc

# Restart
docker-compose up -d server
```

Scenario 2: Complete Server Loss

1. Set up a new server with Docker
2. Clone the repository
3. Copy your `.env` file (especially `BACKUP_ENCRYPTION_KEY`!)
4. Copy your backup files to `./backups/`
5. Run:

```
docker-compose up -d db
# Wait for db to be healthy
./scripts/restore_backup.sh ./backups/your_backup.sql.gz.enc
docker-compose up -d
```

Troubleshooting

Backup service not starting

```
# Check if db is healthy first
docker-compose ps
docker-compose logs backup
```

Cannot restore - wrong key

```
bad decrypt
ERROR: Restore failed!
```

This means the `BACKUP_ENCRYPTION_KEY` doesn't match the key used to create the backup.

Backups are empty (0 bytes)

Check the backup container logs:

```
docker logs caregiver_backup
```

Likely causes:

- Database container not running
- Wrong `POSTGRES_USER` or `POSTGRES_PASSWORD`

Security Notes

1. **Never commit `.env`** to version control (it's in `.gitignore`)
2. **Store the encryption key** separately from backups
3. **Test restores regularly** (at least quarterly)
4. **Offsite backups:** Consider copying `./backups/` to cloud storage for disaster recovery